Team 49

Final Report

AI-Based Early Detection and Forecasting System for Crop Diseases.

(AgriLens)

Team Members:

Nour Osama 202201874 Rahma Abdelwahab 202202059 Merna Ahmed 202201530

Layla Mohammad 202201906

Supervisor: Dr. Mohameds Sami Rakha

Table of Contents

Abstract

Egyptian farmers rely on slow, manual inspections to detect crop diseases, causing delayed diagnosis, rapid spread, and major yield and financial losses. This project proposes an AI-based Early Detection and Forecasting System (AgriLens) that uses fixed and mobile cameras, deep learning models (CNN/YOLO), and cloud-based processing to detect diseases from plant images and predict their spread in near real time. The system combines computer vision, predictive analytics, and services such as AWS, React/Flutter interfaces, and mobile notifications to deliver alerts, maps, and dashboards that highlight disease location, severity, and confidence scores, even in low-connectivity rural settings. The expected value is reduced crop loss, more targeted pesticide use, improved food security, and support for Egypt's shift toward smart, data-driven agriculture through a scalable Detection-as-a-Service model for farmers and agricultural stakeholders.

---

1. Introduction

1.1 Problem Statement

Egyptian farmers, especially those managing small and medium-sized farms, still depend on manual, infrequent field inspections to detect crop diseases, which leads to late diagnosis, rapid disease spread, and substantial yield and income loss. Monitoring relies on human expertise, is time-consuming, and does not scale to large areas, while early disease symptoms are often subtle or invisible until much of the crop is already affected. The

The problem is to design and implement a computing-based system that can continuously monitor crops, detect early signs of multiple plant diseases, and forecast their spread under real-world constraints such as limited

internet connectivity, heterogeneous image quality, and scarce labeled data for Egyptian crops. The scope focuses on

detection and forecasting (not automated treatment), using existing cameras and digital infrastructure, and providing actionable information without requiring farmers to be technology experts.

1.2 Motivation

This problem is important because plant diseases cause large-scale crop losses, financial stress for farmers, increased pesticide usage, and reduced food quality and security at the national level. In Egypt, undetected or late-detected diseases directly threaten rural livelihoods and undermine efforts toward smart, sustainable agriculture. Solving this problem benefits farmers and land owners, who gain early warnings and can take timely interventions; agricultural researchers and agritech companies, who can use data to improve disease management tools; and government and exporters, who rely on healthier crops to meet quality standards and support food security strategies.

1.3 Proposed Solution Overview

The proposed solution is an AI-based Early Detection and Forecasting System (AgriLens) that provides continuous, automated crop health monitoring using images captured from fixed field cameras and farmers' mobile phones. The system will analyze plant images to detect early signs of disease, estimate the likely spread over time and space, and deliver timely alerts and visual summaries through an easy-to-use mobile app and web dashboard. Farmers and stakeholders will see disease locations, severity indications, and prioritized areas of concern, with the option to receive simple, text-based guidance, while the system is designed to function reliably even with intermittent connectivity and without requiring any specialized technical knowledge from the end users.

---

2. Literature Review and Market Survey

2.1 Related Academic / Technical Work

- Plant disease detection using CNNs / YOLO / ViT
    - Recent work on plant disease detection relies heavily on CNN-based classifiers trained on leaf images from datasets such as PlantVillage and its tomato and potato subsets, achieving very high accuracies (often above 95%) in controlled conditions but with limited robustness to field noise, lighting, and local varieties [1]. YOLO-based object detectors extend this by detecting and localizing multiple diseased
    - leaves within images or videos in real time, with YOLOv4 and YOLOv5 variants reporting near-state-of-the-art detection performance on PlantVillage but still focusing mainly on single-image, single-crop scenarios rather than continuous farm-scale monitoring [2]. More recent work explores Vision

  - Transformers (ViT) and hybrid YOLO–ViT pipelines, showing competitive or higher accuracy on benchmark datasets, yet most studies remain limited to offline experiments rather than deployment in production systems integrated with farmers' workflows [3].

- Forecasting approaches (LSTM, ARIMA, Prophet)
  - Forecasting approaches in agriculture use time-series models such as ARIMA, Prophet, and LSTM to predict crop yields, weather variables, or disease risk indices using historical climate, soil, and disease incidence data. ARIMA and Prophet are widely used due to their interpretability and good performance on seasonal patterns [4], while LSTM networks handle non-linear, long-term dependencies and have been applied to spatio-temporal disease severity forecasting and environmental variable prediction. Hybrid and ensemble designs (e.g., ARIMA + Prophet or LSTM + Prophet) often outperform single models, but they are usually evaluated on curated datasets without tight integration into operational decision-support tools for farmers in low-connectivity regions [5].

- Limitations in existing research
  - Most detection models are trained on public datasets from non-Egyptian crops and environments, which reduces generalization to local Egyptian fields without domain adaptation.
  - Many studies assume high-quality, close-up leaf images and ignore issues such as motion blur, partial occlusion, mixed crops, and variable lighting typical in real farms.
  - Forecasting models are usually developed as standalone research prototypes and are not coupled to real-time detection pipelines, alerting mechanisms, or farmer-facing dashboards.

2.2 Existing Systems and Market Solutions

A number of leaf-only detection apps focus on diagnosing diseases from single photos uploaded by farmers. Examples like Plantix turn the smartphone into a "crop doctor" by classifying diseases from leaf images and returning recommended treatments, but they still concentrate on per-image diagnosis without continuous monitoring or explicit forecasting of future risk. Such apps often rely on cloud inference over curated datasets and assume that farmers will manually capture and upload images when symptoms are already visible.

Many commercial and research systems are single-crop or limited-crop solutions, built specifically for tomato, potato, or a few high-value crops, with models and datasets tailored to those plants only. This restricts their applicability to diverse Egyptian farms where multiple crops may coexist in the same field and where farmers expect one tool that covers most of their production, rather than installing separate apps per crop.

Existing tools are also largely reactive rather than predictive: they detect disease once visible symptoms appear but rarely compute forward-looking risk scores based on historical detections, weather forecasts, or environmental conditions. Some

platforms provide disease alerts at regional level, yet these are often coarse, not tightly linked to actual image-based detections on specific fields, and do not exploit models like LSTM or Prophet for farm-level forecasting.

2.3 Comparative Analysis

Aspect	leaf apps (e.g., Plantix, others)	AgriLens
Supported crops	Tens of crops, but often optimized for a few major crops; many tools or datasets focus on single crops like tomato or potato.	Multi-crop design targeting key Egyptian crops with local datasets and domain adaptation.
Input modalities	Manual single photo captures via mobile app; rarely supports fixed cameras or video streams.	Fixed surveillance cameras, smartphone photos, and optional video capture, all feeding a unified pipeline.
Core ML tech stack	CNN-based classifiers on cloud; limited use of forecasting models.	Combined CNN / YOLO / ViT-style detection with LSTM / Prophet / ARIMA forecasting in one architecture.
Features for farmers	Disease classification, basic treatment tips, sometimes community Q&A, and weather info; mainly reactive.	Real-time alerts, interactive web and mobile dashboards, spatial visualization, health history, and optional LLM-based recommendations.
Scalability & deployment	Cloud-hosted but not always optimized for low-bandwidth or offline use; limited integration with external sensors.	Designed for low connectivity (offline caching, delayed sync), scalable cloud processing, and integration with weather and satellite APIs.

2.4 Identified Gap

The literature and market survey indicate that there is currently no integrated, multi-crop, real-time, forecasting-based solution that simultaneously handles continuous image acquisition, leaf-level disease detection, and field-level outbreak forecasting tailored to Egyptian conditions. Existing apps are either strong on detection but limited to reactive single-image diagnosis, or strong on forecasting but exist only as research code without farmer-oriented interfaces or integration with image-based monitoring.

AgriLens fills this gap by:

- Supporting multiple crops and local Egyptian disease varieties through curated datasets and domain adaptation.
- Combining CNN / YOLO / ViT-based detection with LSTM / ARIMA / Prophet forecasting in one pipeline that transforms past detections and environmental data into risk scores and early alerts.
- Providing an end-to-end system from data capture (fixed and mobile cameras) to cloud processing, dashboards, alerts, and optional LLM-generated recommendations, all designed with offline support and scalability via AWS, Docker, RabbitMQ, and Redis.
- Supports video uploads unlike other solutions that supports images only.

3. Requirements Analysis

3.1 Functional Requirements

FR-1: User registration and authentication

The system shall allow farmers and stakeholders to register, log in, and manage their profiles through web and mobile applications.

FR-2: Crop and field management

The system shall allow users to create and manage farm profiles, including fields, crop types, and basic metadata (location, season, etc.).

FR-3: Image acquisition via mobile app

The system shall enable users to capture and upload plant images periodically through a mobile application for inspection.

FR-4: Image ingestion from fixed/surveillance cameras

The system shall receive and store image streams or periodic snapshots from fixed field cameras for continuous monitoring.

FR-5: Video Apploads

The system shall support video apploads so the user can cover large land areas easily the the system extracts important frames to be used in classification and prediction.

FR-6: Image preprocessing pipeline

The system shall preprocess incoming images (resize, normalize, basic quality checks) before passing them to the AI models.

FR-7: AI-based disease detection

The system shall analyze each plant image using AI models to classify plant health status and identify disease type with a confidence score.

FR-8: Disease severity estimation

The system shall estimate and store a severity indicator or risk level associated with each detected disease case.

FR-9: Forecasting of disease spread

The system shall provide a forecasting module that predicts the likelihood and spatial/temporal spread of diseases using historical and environmental data.

FR-10: Integration with weather data

The system shall integrate with a weather API (e.g., OpenWeatherMap) to fetch relevant weather data for use in forecasting models.

FR-11: Integration with satellite/remote sensing data

The system shall allow ingestion of satellite or aerial imagery (e.g., via Google Earth Engine) to support large-scale crop health analysis.

FR-12: Alert generation

The system shall generate alerts whenever a disease (non-healthy class) is detected or when forecasted risk exceeds a defined threshold.

FR-13: Alert delivery via mobile notifications

The system shall send alerts to users through mobile push notifications using a notification service (e.g., Firebase Cloud Messaging).

FR-14: Alert delivery via SMS (if configured)

The system shall support sending SMS alerts where required or configured by the user or organization

FR-15: Web dashboard visualization

The system shall provide a web dashboard showing disease cases, field status, severity levels, confidence scores, and trends.

FR-16: Map-based visualization

The system shall visualize disease locations on a map view (per field/region) for spatial awareness.

FR-17: Mobile health monitoring view

The system shall provide a simplified mobile view showing current plant health status, recent detections, and key alerts for a farmer’s fields.

FR-18: Historical data browsing

The system shall allow users to view historical detections, trends, and past disease events for selected fields and time ranges.

FR-19: Reporting and summaries

The system shall generate summary reports (e.g., weekly/monthly) of detections, forecasted risks, and basic performance statistics for a user's farms.

FR-20: Dataset management for model training

The system shall store and organize labeled plant images and associated metadata for training, validation, and testing of AI models.

FR-21: Model evaluation and metrics logging

The system shall compute and log core performance metrics (accuracy, precision, recall, F1-score) on validation/test data for each model version.

FR-22: Model versioning and deployment

The system shall support deploying updated model versions while tracking which version is used for each prediction.

FR-23: Offline data capture on mobile

The mobile app shall allow image capture and local caching when the internet is unavailable, then synchronize data when connectivity is restored.

FR-24: Synchronization and conflict handling

The system shall synchronize offline-captured data with the cloud backend and resolve basic conflicts (e.g., duplicate uploads).

FR-25: Subscription and access control for Detection-as-a-Service

The system shall support subscription-based access for farmers/agri-businesses, enabling or restricting features according to subscription level.

FR-26: Basic recommendation messages

The system shall provide simple, non-prescriptive, text-based suggestions or guidelines associated with detected diseases and forecasted risks; optionally generated or enhanced via LLMs.

FR-27: System monitoring and logging

The system shall log key events (detections, alerts, failures, latency measures) and expose them for monitoring and troubleshooting.

FR-28: Security and privacy controls

The system shall enforce authentication, authorization, and data access rules; ensure that no personally identifiable information of individuals is stored from field images.

FR-29: Team collaboration and project artifacts

The project team shall maintain source code in a version control system (e.g., Git/GitHub), track issues/tasks, and keep project documentation updated.

3.2 Non-Functional Requirements

Performance

NFR-P1: Detection latency

The system shall process each incoming image and return a detection result within less than 10 seconds seconds under normal operating conditions

NFR-P2: Alert delivery time

The system shall deliver alerts (notification or SMS) to the user within seconds after a disease case is detected.

NFR-P3: Dashboard responsiveness

The web dashboard and mobile views shall load core health information (current field status and recent alerts) within 3seconds on an internet connection.

NFR-P4: Forecasting update frequency

The system shall update forecasting outputs (disease spread risk) at least once per configured time window (e.g., hourly or daily), depending on data availability.

Security

NFR-S1: Data encryption

All communication between clients (mobile/web) and backend services shall be encrypted in transit using industry-standard protocols (e.g., HTTPS/TLS).

NFR-S2: Privacy compliance

The system shall avoid collecting personally identifiable information about individuals within camera views and comply with relevant Egyptian data protection regulations.

Scalability

NFR-SC1: Horizontal scaling of services

The backend and AI inference components shall be deployable on scalable cloud infrastructure (e.g., microservices on AWS) to handle increasing numbers of farms, cameras, and users.

NFR-SC2: Dataset growth handling

The system shall support growth in stored images and metadata (e.g., multi-season, multi-crop data) without significant degradation in query and retrieval performance.

NFR-SC3: Multi-region expansion

The architecture shall allow extension to new geographic regions, crops, and disease types with minimal changes to core services.

Usability

NFR-U1: Dashboard intuitiveness

Farmers and non-technical stakeholders shall be able to interpret disease locations, severity, and basic trends from the dashboard without formal training.

NFR-U2: Mobile simplicity

The mobile app shall enable users to capture and submit an image in no more than three main steps (open camera, capture, submit).

NFR-U3: Alert clarity

At least 90% of test users shall correctly understand the meaning of alert severity levels and recommended actions in usability evaluations.

NFR-U4: Localization

The user interface shall support at least Arabic and English to ensure accessibility for Egyptian farmers.

Reliability

NFR-R1: System availability

The system shall maintain at least 95% uptime during the main agricultural season, excluding planned maintenance windows.

NFR-R2: Offline operation

The mobile application shall continue to function for image capture and local storage when internet connectivity is lost and synchronize automatically when connectivity is restored.

NFR-R3: Data integrity

The system shall ensure no loss of stored images, predictions, or alert logs during normal operation, with mechanisms for safe retries on failed uploads.

NFR-R4: Fault tolerance and recovery

The system shall recover from component failures (e.g., service restarts, transient network issues) without data corruption and with minimal interruption to monitoring and alerting.

---

4. System Design

4.1 Overall System Architecture

- System Architecture.jpg

4.2 Component Breakdown

4.2.1 Data Sources

- **Function:** Provide raw data for detection and forecasting.
- **Inputs:** Smartphone/drone images; weather and climate data; historical disease reports.
- **Outputs:** Raw image files; historical datasets.
- **Main Functionality:** Collect heterogeneous data and stream it into the preprocessing pipeline.
- **Technologies:** Camera hardware, OpenWeather API, simple HTTP upload endpoint.
- **Justification:** Low-cost, easy to integrate, standard for agricultural monitoring workflows.

4.2.2 Preprocessing Pipeline

- **Function:** Clean and prepare raw images and datasets.

- **Inputs:** Raw images, metadata, weather datasets.
- **Outputs:** Normalized, resized images stored in object storage.
- **Main Functionality:** Resize to model input size, normalize channels, perform quality checks, log results.
- **Technologies:** Python, NumPy, Pandas, OpenCV, Scikit-learn.
- **Justification:** Widely used, stable toolchain with optimized image processing and data cleaning performance.

4.2.3 Disease Detection Model (AI Model)

- **Function:** Identify disease type and severity from images.
- **Inputs:** Preprocessed image tensors.
- **Outputs:** Disease label, confidence, bounding boxes, healthy/unhealthy flag.
- **Main Functionality:** CNN/ViT inference, thresholding, prediction logging.
- **Technologies:** TensorFlow/PyTorch, pretrained CNN/ViT models, CUDA.
- **Justification:** State-of-the-art vision models delivering high accuracy with optimized training/inference speed.

4.2.4 Forecasting Model

- **Function:** Predict future disease risk and potential spread.
- **Inputs:** Past detections, weather forecasts, sensor time-series, historical outbreak data.
- **Outputs:** Risk score, short-term time-series forecast, spatial risk map.
- **Main Functionality:** LSTM/Prophet-based temporal modeling, multivariate pattern analysis, probabilistic output generation.
- **Technologies:** PyTorch/TensorFlow, Prophet/ARIMA, NumPy, Pandas.
- **Justification:** Mature frameworks that handle multivariate forecasting reliably and efficiently.

4.2.5 Central Database / Storage

- **Function:** Store all system data and outputs.
- **Inputs:** Images, detection results, forecasts, uploads.
- **Outputs:** Queryable datasets for backend and models.
- **Main Functionality:** Persist structured and unstructured data, index queries, maintain temporal logs.
- **Technologies:** MongoDB, Firebase Storage, AWS S3.
- **Justification:** Scales well for mixed data types and supports fast access patterns for analytics and ML.

4.2.6 Backend API Layer

- **Function:** Bridge between frontend, models, and database.
- **Inputs:** User/API requests; model outputs; stored data.
- **Outputs:** JSON responses, graphs, alerts.
- **Main Functionality:** REST endpoints, auth handling, image uploads, routing to inference services.
- **Technologies:** Flask, TorchServe/TensorFlow Serving, Docker, cloud deployment.
- **Justification:** Lightweight, Python-native stack ideal for ML serving and scalable deployment.

4.2.7 User Dashboard

- **Function:** Provide visualization and interaction for users.
- **Inputs:** API results, user uploads.
- **Outputs:** Detection summaries, risk timelines, maps, analytics.
- **Main Functionality:** Display predictions, embed charts, plot spatial data, enable uploads.
-
- **Technologies:** React, ChartJS/Plotly, Leaflet, Netlify.
- **Justification:** Supports responsive, interactive UI with powerful visualization libraries.

4.2.8 Notification / Alert System

- **Function:** Trigger warnings when risk is high.
- **Inputs:** Forecast outputs, backend threshold rules.
- **Outputs:** SMS, push, email alerts.
- **Main Functionality:** Threshold evaluation, message automation, near real-time notifications.
- **Technologies:** Twilio, Firebase Cloud Messaging.
- **Justification:** Reliable messaging infrastructure with easy Python integration.

4.2.9 Mobile App (Farmer Client)

- **Function:** Enable image capture, uploads, and alert viewing.
- **Inputs:** Images, metadata, alerts.
- **Outputs:** Upload requests; UI views showing results and recommendations.
- **Main Functionality:** Local validation, offline caching, capture-upload-view workflow.

- **Technologies:** Flutter, Firebase SDK.
- **Justification:** Fast cross-platform development suitable for widespread Android/iOS use.

4.2.10 Messaging & Caching Layer

- **Function:** Decouple services and speed up repeated queries.
- **Inputs:** Backend events; frequent dashboard queries.
- **Outputs:** Delivered events and cached responses.
- **Main Functionality:** Pub/sub messaging, short-term caching, low-latency access.
- **Technologies:** RabbitMQ, Redis.
- **Justification:** Reliable ecosystem with strong support for distributed systems and ML backends.

4.2.11 External Data & Integration Services

- **Function:** Provide weather, satellite imagery, and optional LLM-based insights.
- **Inputs:** Requests from forecasting or recommendation services.
- **Outputs:** Weather series, remote sensing indices, generated text suggestions.

- **Main Functionality:** API calls, normalization, feeding external data into internal models.
- **Technologies:** OpenWeather API, Google Earth Engine, LLM API.
- **Justification:** High-quality external datasets and flexible integration for advanced analytics.

4.3 Design Decisions and Rationale

The system uses a microservices architecture with MVC (Model-View-Controller) and observer patterns to separate ingestion, AI inference, forecasting, notifications, and UI, enabling independent deployment, easier scaling, and event-driven alerts and updates. This structure supports continuous enhancement of models and services without disrupting the overall platform, which is essential for a long-lived Detection-as-a-Service solution.

A cloud-first deployment on AWS is chosen to leverage elastic compute, storage, and monitoring, reducing infrastructure overhead while supporting growth in farms, cameras, and images. This aligns with the need for high availability and multi-tenant access for farmers, agribusinesses, and authorities.

Python is selected for AI development (TensorFlow/PyTorch, NumPy, Pandas), with Flask/Node.js for lightweight, API-based backends and React/Flutter for modern web and mobile interfaces, balancing rapid development with

cross-platform reach. MongoDB, Firebase, RabbitMQ, and Redis are used together to handle flexible data schemas, real-time sync, reliable service-to-service messaging, and caching of frequently accessed status and metrics.

CNN/YOLO-based models, supported by Roboflow and OpenCV, are adopted for high-accuracy image-based disease detection and practical deployment on real farm imagery. Integrations with OpenWeatherMap and Google Earth Engine add weather and satellite data, improving forecasting of disease spread beyond what single-image analysis can provide.

Finally, an offline-capable, bilingual (Arabic/English) UI is prioritized so farmers in low-connectivity rural areas can still capture images and later sync, and so non-technical users can interpret alerts and dashboards with minimal steps and no formal training.

4.4 Program-Specific SO(6) Focus (MANDATORY)

4.4. SWD – Software Development

The system will follow Microservices **Architecture**, **MVC,** and **Observer** as design patterns.

- **Microservices:** It will allow independent module operation, scaling, and easier maintenance.

- **MVC (Model-View-Controller):** Separates data logic, user interface, and flow controllers, improving code clarity and maintainability.
- **Observer Pattern:** Utilizes real-time updates and alerts by notifying connected components when events such as detections or forecasts happen.
- **Service-oriented design** to enable loose coupling between independent system components.

4.4. DSAI – Data Science & AI

• Data lifecycle (collection → preprocessing → modeling → evaluation)

4.4.1 Data Flow diagram

4.4.2

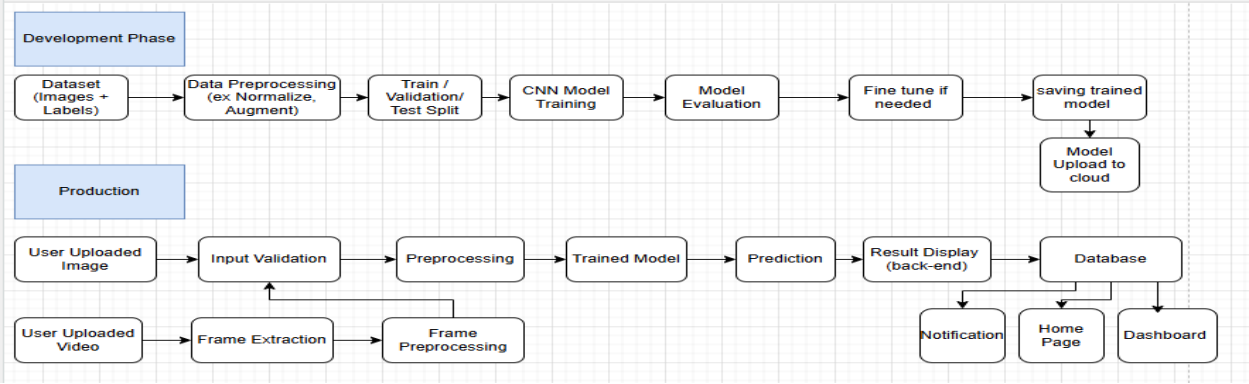

Figure 1. *Deep Learning Data Flow*

• ML / AI models and justification

CNN/YOLO-based computer vision models are used for leaf disease detection because they provide high accuracy and support near real-time inference on image streams. A separate predictive module uses historical detections and environmental variables to forecast disease spread and recurrence, matching the need for both instant detection and proactive risk estimation.

• Feature engineering and evaluation metrics

Detection relies on image features (color, texture, lesion patterns) learned by deep networks plus metadata like crop type and location, while forecasting uses time-based aggregates of detections, weather, and regional indicators. Evaluation uses accuracy, precision, recall, F1-score, forecasting accuracy, early detection rate, and false alarm rate, with targets such as 90% accuracy and ≤10% false alarms to ensure trustworthy results.

• Model integration into the system

Preprocessed images flow to an inference service that returns disease class, confidence, and severity, which are stored, visualized on dashboards, and used to trigger alerts. A separate forecasting service periodically reads stored data and updates risk predictions, with both services deployed as cloud containers to enable safe versioning and continuous improvement.

---

5. Project Timeline (Next Semester)

Phases and milestones

- Current – Design and Planning (Week 10–12)

  - Finalize system architecture and UI/UX design.
  - Produce detailed architecture diagram (frontend–backend–database–ML pipeline).
  - Define all data sources (images, metadata, labels, weather data).
- Phase 1 – Development (Week 15–20)
  - Implement prototype model for image classification.
  - Build basic web dashboard prototype.
  - Develop data preprocessing pipeline.
  - Select and implement model architectures; perform initial error analysis.
  - Generate test reports and evaluation metrics for the prototype.
- Phase 2 – Testing (Week 21–24)
  - Conduct model fine-tuning on local Egyptian data.
  - Run extended tests and report detailed evaluation metrics.
  - Summarize prototype performance.
  - Perform usability testing with real farmers or classmates and refine the system based on feedback.

- **Responsibilities per member**

Task	Responsible Member
Integration with AI model & forecasting	Nour + Layla
Web dashboard implementation (photo + video)	Merna + Layla
Mobile app implementation (photo + video capture & upload)	Merna + Rahma
System integration tests	Layla + Rahma
Performance & reliability tests	Nour + Rahma

User acceptance testing (farmers)	Layla + Nour
Evaluation & final report	Whole Team (lead: Nour)
User manual & technical documentation	Whole Team

- gantt.pdf

---

6. Challenges and Solutions

Challenge category	**Challenge Description**	**Mitigation Strategy**
Technical	Datasets that are available online have various qualities, balance levels, and relevance to Egyptian farming conditions, which could affect the models' performance	Make an early exploratory analysis (EDA), and apply augmentation techniques to improve performance
Technical	Video processing is too slow (frame extraction + inference)	Limit video size and resolution, sample frames periodically, use lightweight models (e.g., YOLO-tiny, pruning, quantization), and shift heavy processing to monitored GPU-cloud instances.

Technical	API limitations or instability (image/video upload, inference endpoints)	Define API contracts early, version the endpoints, add client-side retries/timeouts, and set up automated integration tests for image/video flows before UAT.
Technical	Mobile app instability on low-end devices	Test on low-end Android devices, optimize capture/compression, minimize memory usage, and add crash logging to quickly identify device-specific issues.
Resource	Limited labeled data for specific disease classes	Mix public datasets with locally collected data, prioritize labeling rare/critical diseases, use semi-supervised/augmentation methods, and have experts periodically verify labels.
Resource	Low farmer engagement in UAT (few real test cases)	Coordinate early with supervisors/partners to recruit farmers, schedule UAT ahead of time, prepare simple scenarios and incentives, and
Timeline	Poor internet connectivity in rural areas is affecting UAT and deployment	Implement and test offline mode with local caching, delayed or batch syncing, compressed uploads, and

		alternative connectivity options for on-site pilots.
Organizational	Dividing tasks across a diverse major team with different backgrounds	Divide tasks based on the major and give each person a suitable task that fits best with their skills

---

7. Work Summary (Fall Semester)

We finished the UI wireframes for the mobile app, which included notifications, results display, upload, and image capture. In order to get ready for model training, we also completed the first EDA for the tomato dataset by examining sample photos and class distribution. Graduation Project UI DEMO

Comprehensive research [6] was done exploring different research papers and datasets[7], especially for tomatoes and potato crops, to deepen the understanding of the project scope, and multiple models were explored [8] to be taken into consideration.

Tomato crops were selected due to their economic importance and high susceptibility to fungal, bacterial, and viral diseases. A literature and dataset review identified common tomato diseases such as early blight, late blight, Septoria leaf spot, and tomato mosaic virus. These diseases present visually distinguishable symptoms on leaves, making them suitable for image-based detection using computer vision techniques.

A publicly available tomato leaf disease dataset was identified [9], containing labeled images of healthy and diseased tomato leaves. The dataset includes multiple disease classes and high-resolution RGB images, making it suitable for training and evaluating deep learning-based image classification models.

Based on the dataset characteristics and disease symptoms, an image-based disease classification approach was selected. The system is designed to accept leaf images as input, perform preprocessing and feature extraction, and output the predicted disease class along with confidence scores. This design supports future integration with forecasting and decision-support modules.

Initial coding was performed in a Python Colab notebook added to the project's repository [10] to explore the tomato leaf disease image dataset, which consists of ten classes, including nine disease categories and one healthy class. The

exploratory analysis examined class distribution and sample images, revealing variability in the number of images per class, with Tomato Yellow Leaf Curl Virus being the most frequent. Visual inspection confirmed that each disease exhibits distinct leaf symptoms, supporting the feasibility of training an image-based disease classification model.

Due to the lack of knowledge about the genetic diversity of S.scabies and the genetic variations among different potato cultivars, control methods for potato scab are difficult.

Chemical control techniques, cultural strategies like crop rotation, and traditional management methods like reducing soil pH and increasing irrigation intensity are not usually effective. [11] Therefore, the potato is chosen as one of the crops whose diseases our system will detect in early stages.

Two deep learning architectures were applied to practice frame extraction, Model 1: Convolutional Neural Network + Temporal Convolutional Network (CNN + TCN) and Model 2: ResNet50 + Conv1d
Preliminary results indicate that both models assign high keyframe scores across the video, indicating continuous motion.
The CNN + TCN model shows sharper fluctuations, reflecting higher sensitivity to sudden temporal changes, however it can assume redundant frames as unique.
The ResNet + Conv1D model produces slightly more stable and smoother scores, suggesting stronger temporal consistency but may miss minimal change; however it also has sudden spikes and fluctuations.

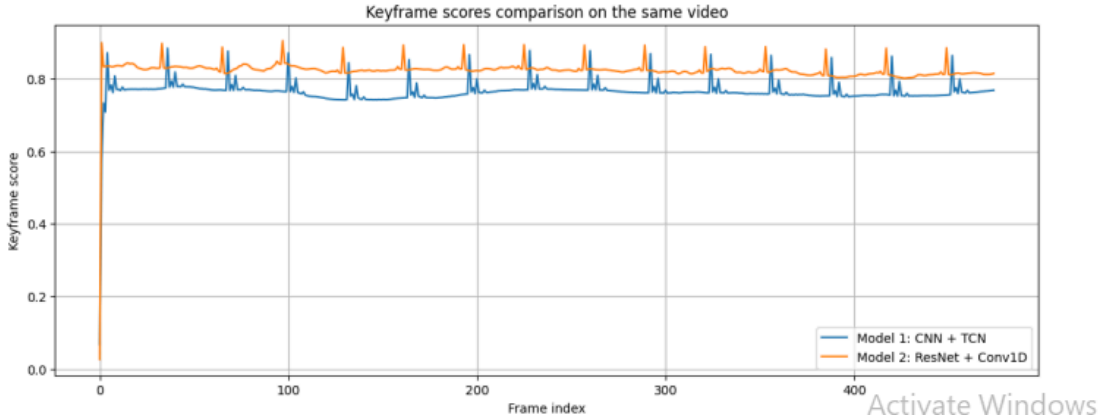

***Fig2**. comparing models on 37 secondes video*

Planning to test models on longer videos and to try other model architectures.

Individual Team Member Contributions

Merna Ahmed(SWD):

- Designed UI/UX wireframes.
- Contributed to designing the system architecture, which includes a microservices structure and MVC-based organization.

Nour Osama:(DSAI):

- Conducted research on relatable academic work, datasets, and existing disease detection approaches for tomato and potato crops.
- Conducted exploratory data analysis (EDA) for the tomato dataset, including sample inspection and class distribution analysis.

- Contributed to crop selection rationale, focusing on tomato and potato due to disease prevalence and economic importance.

Rahma Abd El-Wahab(DSAI):

- Participated in literature review and dataset exploration for tomato and potato diseases.
- Conducted initial exploratory data analysis (EDA) on the tomato leaf disease dataset, examining class balance and visual disease characteristics.
- Co-designed and finalized mobile application UI wireframes
- Prepared the project schedule and Work Breakdown Structure (WBS).

Layla Zayed(DSAI)

- Participated in research on datasets and disease characteristics, particularly for tomato and potato crops.
- Contributed to exploratory data analysis (EDA) by reviewing sample images and validating dataset suitability for model training.
- Designed and documented the Deep Learning Data Flow diagram, illustrating both training and production inference pipelines, including image/video input handling, preprocessing, model inference, result storage, and notification flow.
- Applied two deep learning models for frame extraction from videos and compared their results.

References

[1] R. K. Singh et al., “Plant leaf disease detection using vision transformers for smart agriculture,” Sci. Rep., vol. 15, 2025. [Online]. Available: https://www.nature.com/articles/s41598-025-05102-0

[2] M. M. M. El-Sayed et al., “Detection and identification of plant leaf diseases using YOLOv4,” Plant Methods, vol. 20, no. 1, 2024. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC11070553/

[3] A. S. Al-Waisy et al., "Ensembling YOLO and ViT for plant disease detection," in Proc. Int. Conf. Adv. Mach. Learn. Technol. Appl., 2024. [Online]. Available: https://dl.acm.org/doi/10.1007/978-3-031-78305-0_6

[4] S. T. Nguyen and H. T. Le, "Automated hydroponic growth simulation for lettuce using ARIMA," in Proc. Int. Conf. Adv. Res. Artif. Intell., 2023. [Online]. Available: https://thesai.org/Downloads/Volume15No11/Paper_37-Automated_Hydroponic_Growth_Simulation.pdf

[5] "Crop prediction using soil metrics with LSTM and Prophet," Int. J. Res. Eng. Innov., vol. 8, no. 4, pp. 322–330, 2024. [Online]. Available: https://ijrei.com/assets/frontend/aviation/1746551778_bcc98a9dae70c0e11797.pdf

[6] S. Y. Prasetyo, "Overcoming overfitting in CNN models for potato disease classification using data augmentation," *Eng. Math. Comput. Sci. (EMACS) J.*, vol. 6, no. 3, pp. 179–184, Sep. 2024, doi: https://doi.org/10.21512/emacsjournal.v6i3.11840

[7] N. H. Shabrina, S. Indarti, R. Maharani, D. A. Kristiyanti, Irmawati, N. Prastomo, and T. A. Me, "A novel dataset of potato leaf disease in an uncontrolled environment," *Data Brief*, Dec. 2023, doi: https://doi.org/10.1016/j.dib.2023.109955.

[8] G. Sangar and V. Rajasekar, "Optimized classification of potato leaf disease using EfficientNet-LITE and KE-SVM in diverse environments," *Front. Plant Sci.*, vol. 16, May 2025, doi:https://doi.org/10.3389/fpls.2025.1499909.

[9] N. Gull, "Tomato Leaf Disease Dataset," *Kaggle, 2023*. [Online]. Available. [Accessed: 13-Dec-2025].

[10] Project source code, private Git repository, 2025.

[11] O. A. Abd El-Hafez and A. F. Abd El-Rahman, "Possible control of potato common scab with indigenous nonpathogenic species of Streptomyces in Egypt," *Egypt. J. Phytopathol.*, vol. 47, no. 1, pp. 79–95, 2019. doi: https://doi.org/10.21608/ejp.2019.120015